

The Residual Stream

Every layer adds; nothing is replaced · video 2:51

This lesson answers one question: how do the layers of a transformer pass information along? Not by replacing it: each attention and MLP block **adds** its output to a running vector, the **residual stream**. In GPT-2, most updates are 15–34% of the stream's size, so deleting a middle layer barely matters; without the addition, an 8-layer tiny model cannot learn anything beyond letter frequencies.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. A stream that every layer writes into	▶ 0:08-0:36	2	☐
2. How big it is, and how big the edits are	▶ 0:36-1:20	2	☐
3. Delete a layer; remove the stream	▶ 1:20-2:16	2	☐
4. The gradient highway, and the code	▶ 2:16-2:40	2	☐

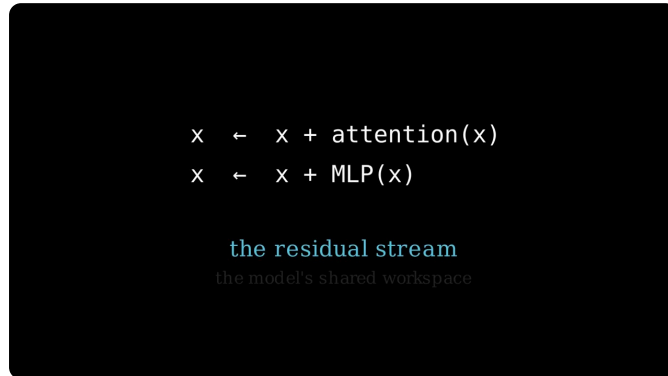
Before you start: How LLMs Work episodes 7–8 (the transformer block) and Foundations (gradients). Code: `code/d13_residual_stream` (GPT-2 small, about 500 MB, plus two tiny models, about 10 minutes on a CPU).

CONCEPT 1 OF 4

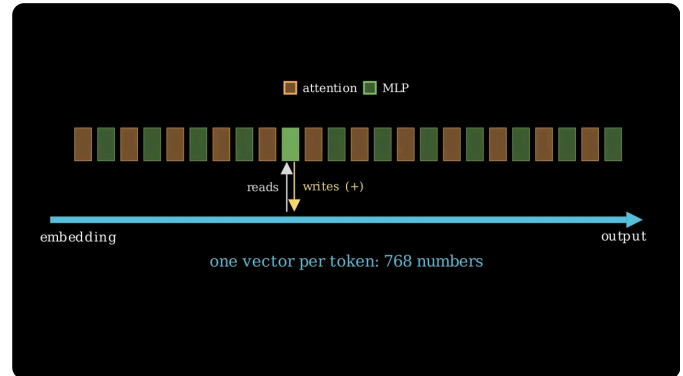
A stream that every layer writes into

▶ Video 0:08-0:36

Inside a transformer, no layer replaces what came before. Each one adds: $x = x + \text{attention}(x)$, then $x = x + \text{MLP}(x)$. The running sum is the **residual stream**, the model's shared workspace.



▶ 0:20 $x \leftarrow x + \text{attention}(x)$, then $x \leftarrow x + \text{MLP}(x)$: the running sum is the residual stream.



▶ 0:35 One 768-number vector per token flows from embedding to output; 24 sub-blocks read it and write corrections.

Picture one vector per token (768 numbers in GPT-2) flowing from the embedding to the output. Along the way sit 24 sub-blocks, an attention and an MLP in each of the 12 layers. Each one **reads** the stream (through a normalization) and **writes** a correction back into it.

KEY IDEA

The residual stream is a sum: embedding + every block's output. Each block edits it; none overwrites it.

Check 1: A stream that every layer writes into

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

How many sub-blocks write into GPT-2 small's residual stream?

.....

1.2 SHORT ANSWER

Write the residual stream after two layers as a sum of terms.

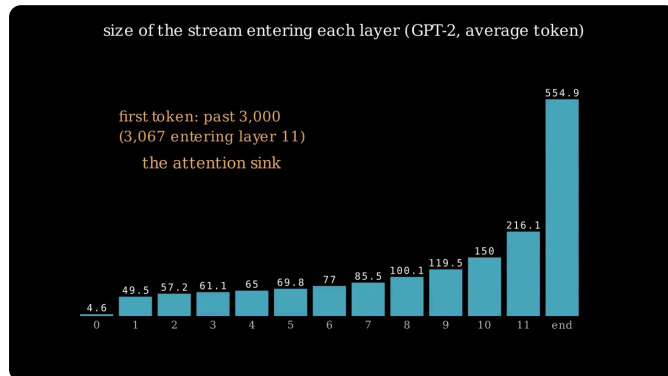
.....

CONCEPT 2 OF 4

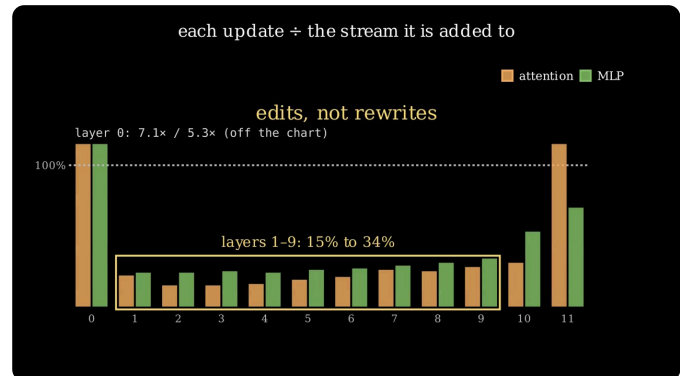
How big it is, and how big the edits are

▶ Video 0:36–1:20

In GPT-2 the stream starts small: an average size of **4.6**, just the embedding. After layer 0 it is about 50, then it grows steadily to **216** entering the last layer and 555 at the end. The first token is different: its stream grows past **3,000** (3,067 entering layer 11), the attention sink of episode 12, a huge vector every head can find.



▶ 0:59 The stream's size entering each layer: 4.6 (embedding), 49.5 after layer 0, up to 216.1 and 554.9 at the end.



▶ 1:19 Each update as a fraction of the stream: 15–34% in layers 1–9; layer 0 and the last layers write more.

Compared with the stream they are added to, the updates in layers 1–9 are only **15% to 34%** of its size: edits, not rewrites. Layer 0 writes 7.1× (attention) and 5.3× (MLP) the embedding: it builds the stream. The last layers make bigger changes (up to 1.41×) to prepare the output.

KEY IDEA

Most layers nudge the stream; the first builds it and the last ones reshape it for the output.

Check 2: How big it is, and how big the edits are

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

By what factor does the average token's stream grow from entering layer 1 (49.5) to entering layer 11 (216.1)?

2.2 TRUE OR FALSE

"In GPT-2's middle layers, each MLP replaces most of the stream."

True ☐ False ☐

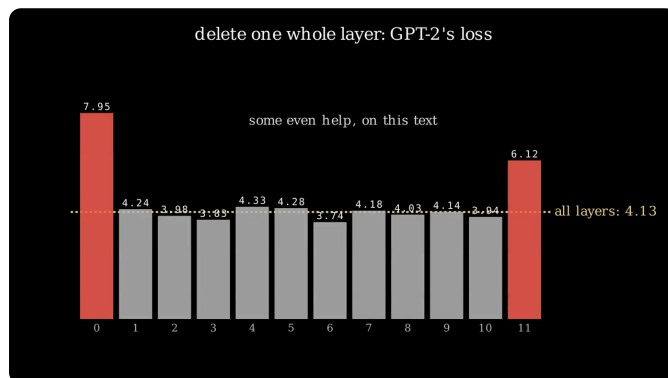
CONCEPT 3 OF 4

Delete a layer; remove the stream

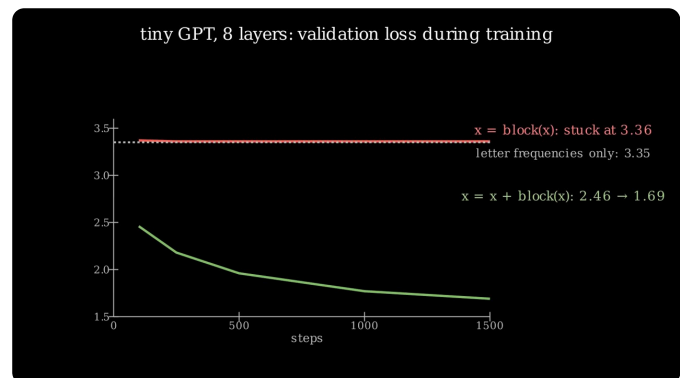
▶ Video 1:20-2:16

Delete a whole layer from GPT-2, letting the stream flow past it. The baseline loss on this Shakespeare text is **4.13**. Without any one middle layer, the loss stays between **3.74 and 4.33**; on this text some deletions even help. Without the first layer: **7.95**; without the last: **6.12**. Because each layer only adds, deleting one removes one edit; in a plain chain, every layer would depend on the exact output of the one before.

Now train the same 8-layer tiny GPT with $x = x + \text{block}(x)$ or just $x = \text{block}(x)$. With the residual: 2.46 after 100 steps, **1.69** after 1,500. Without it: stuck at **3.36** from the start, almost exactly the 3.35 you get by predicting from letter frequencies alone. It learned nothing else.



▶ 1:38 Delete one layer: middle layers 3.74-4.33 (baseline 4.13); the first 7.95, the last 6.12.



▶ 2:15 8-layer tiny GPT: with residuals 2.46 → 1.69; without, stuck at 3.36 (letter frequencies alone: 3.35).

KEY IDEA

Residual connections make layers robust to removal and, above all, make deep networks trainable.

Check 3: Delete a layer; remove the stream

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

Which deletion hurt GPT-2 most on this text?

- A. Layer 6
- B. Layer 0
- C. Layer 11
- D. Layer 4

3.2 SHORT ANSWER

The model without residuals reached 3.36, almost exactly the letter-frequency loss (3.35). What does that tell you about what it learned?

CONCEPT 4 OF 4

The gradient highway, and the code

▶ Video 2:16-2:40

Why does training fail without the addition? The derivative of $x + f(x)$ is $1 + f'(x)$. The 1 carries the learning signal straight back through all the blocks, so early layers get a usable gradient however deep the network is. Without it, the gradient must pass through every block's derivative in turn, and can shrink or scramble on the way.

$$d/dx [x + f(x)] = 1 + f'(x)$$



▶ 2:28 $d/dx [x + f(x)] = 1 + f'(x)$: the 1 carries the learning signal straight back to the early layers.

```
1 def forward(self, x):
2     x = x + self.attn(self.ln1(x)) # attention adds to the stream
3     x = x + self.mlp(self.ln2(x)) # the MLP adds to the stream
4     return x
5
6 # without the stream (the experiment):
7 # x = self.attn(self.ln1(x)); x = self.mlp(self.ln2(x))
```

▶ 2:39 Two plus signs; removing them was the whole experiment.

KEY IDEA

The residual stream is also a gradient highway: the identity path lets every layer learn.

Check 4: The gradient highway, and the code

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

Suppose each of 8 blocks multiplies the gradient by 0.5. What fraction reaches the first block without residuals? With residuals, at least what comes through the identity path?

4.2 HANDS-ON CODE

Try it yourself. With `run` from `residual_stream.py`, delete layers 3 and 6 together, then layers 2, 3, 6 and 10. Is the damage additive?

```
for skip in ({3, 6}, {2, 3, 6, 10}):
    lg, _, _ = run(ids, skip=skip)
    print(sorted(skip), F.cross_entropy(lg[0, :-1], ids[0, 1:]).item())
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. A stream that every layer writes into · rewatch ▶ 0:08-0:36

1.1 24. 12 layers $\times$ (1 attention + 1 MLP).

1.2 $x_2 = \text{embedding} + \text{attn}_0 + \text{mlp}_0 + \text{attn}_1 + \text{mlp}_1$ (each computed from the stream as it was at that point). Every block adds its output; nothing is subtracted or replaced.

2. How big it is, and how big the edits are · rewatch ▶ 0:36-1:20

2.1 About 4.4. $216.1 / 49.5 \approx 4.37$.

2.2 False. Its output is about a quarter of the stream's size (0.24-0.34 in layers 1-9), and it is added, not substituted.

3. Delete a layer; remove the stream · rewatch ▶ 1:20-2:16

3.1 B. Loss 7.95 without layer 0, against 6.12 without layer 11 and 3.74-4.33 for middle layers.

3.2 Only how common each character is, ignoring the context entirely: its output does not depend on the input in any useful way. A model that predicts the same distribution everywhere scores exactly the unigram loss.

4. The gradient highway, and the code · rewatch ▶ 2:16-2:40

4.1 $0.5^8 \approx 0.0039$ without; the full gradient (factor 1) through the identity path with residuals. Products of small factors vanish; the residual adds a path whose factor is exactly 1.

4.2 Layers 3 and 6: 3.71, lower than either alone (3.83, 3.74). Four middle layers: 4.32, still close to the baseline 4.13. Not additive: the remaining layers' edits interact. But with residuals, removing several middle layers degrades gracefully on this text.